

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет «Радиотехнический»
Кафедра «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»

**Отчет по лабораторной работе №4-5
«Программирование на языке Rust»**

Выполнил:
студент группы РТ5-31Б:
Кузнецов С. А.

Подпись и дата:

Проверил:
преподаватель кафедры ИУ5
Гапанюк Ю. Е.

Подпись и дата:

Москва, 2024 г.

Постановка задачи

Разработать программу, реализующую вычисление расстояния Левенштейна с использованием алгоритма Вагнера-Фишера. Дополнительно реализовать вычисление расстояния Левенштейна-Дамерау. Программа должна содержать модульные тесты и хотя бы один макрос.

Текст программы

Файл «main.rs»:

```
//Нужен Левенштейн с модульными тестами и хотя бы одним макросом

mod levenshtein;

use std::io::{
    stdin,
    stdout,
    Write
};

/// Чтение одной строки из консоли. По идее должно работать корректно на
Windows и Linux.
/// Возвращает единичку, потому что... :3
fn read_input(buf: &mut String) -> usize
{
    let _ = stdout().flush(); // Убедимся, что в потоке ничего нет
    stdin()
        .read_line(buf)
        .expect("Invalid input");

    // Очищаем... trailing characters, хз как по-русски.
    // /r/n - Windows, /n - Unix
```

```

    if let Some('\n') = buf.chars().next_back() {
        buf.pop();
    }
    if let Some('\r') = buf.chars().next_back() {
        buf.pop();
    }

    1
}

fn main() {
    println!("Please enter the first UTF-8 string.");
    let mut s1 = String::new();

    read_input(&mut s1);

    let mut s2 = String::new();

    println!("Please enter the second UTF-8 string.");

    read_input(&mut s2);

    let distance = levenshtein::l_distance(&s1, &s2);
    let distance2 = levenshtein::s_d_distance(&s1, &s2);

    println!("Levenshtein distance between {s1} and {s2} is {distance}.");
    println!("Damerau-Levenshtein distance between {s1} and {s2} is {distance2}.");
}

```

Файл «levenshtein.rs»:

```
/// Макрос, возвращающий наименьшее число из произвольного числа переданных.
```

```
/// Возможно, не очень дружит с числами разных типов...
```

```
macro_rules! min_of_n {
```

```
    // (Я уверен, что где-то в стандартной библиотеке такое уже есть)
```

```
    // Не уверен, что код снизу "работает", но вроде как попытка вызвать макрос
```

```
    // без аргументов вынуждает компилятор ругаться, что хорошо :)
```

```
    () => {
```

```
        panic!("Attempted to find the smallest number in an empty number set")
```

```
    };
```

```
    ($($args:expr),*) => {
```

```
        {
```

```
            let mut smallest = usize::MAX;
```

```
            $(
```

```
                if smallest > $args
```

```
                {
```

```
                    smallest = $args;
```

```
                }
```

```
            )*
```

```
            smallest
```

```
        }
```

```
    };
```

```
}
```

/// Функция находит расстояние Левенштейна между двумя строками алгоритмом Вагнера — Фишера.

```
pub fn l_distance(s1: &str, s2: &str) -> usize
```

```
{
```

// &str говорит о том, что входные строки лишь читаются, внешний контекст сохраняет владение ими.

// Для Rust неверно предполагать, что символ кодируется одним байтом, поэтому длина строки считается не

// функцией len(). Кроме того, невозможна индексация вида s1[i]...
Грустно!

```
let s1len = s1.chars().count();
```

```
let s2len = s2.chars().count();
```

```
if s1len == 0 && s2len == 0
```

```
{
```

```
    return 0;
```

// Тривиальный случай: обе строки пустые. Не требует вычисления D.

```
}
```

```
else if s1len == 0 && s2len > 0
```

```
{
```

```
    return s2len;
```

// Тоже тривиально: чтобы воспроизвести непустую строку в пустой, нужно

// последовательно вставить в неё все символы непустой строки.

```
}
```

```
else if s2len == 0
```

```
{
```

```

return s1len;

// Последний тривиальный случай аналогичен второму.
}

let cell_count = (s1len + 1) * (s2len + 1); // Общее число ячеек матрицы
let mut d: Vec<usize> = vec![0; cell_count]; // Матрица D в виде одномерного
вектора

for i in 0..(s1len+1)
{
    d[i * (s2len + 1)] = i;
}

for j in 1..(s2len+1)
{
    d[j] = j;
}

// Если мы имеем невырожденный случай, то клетки первого столбца и
первой строки
// ("первой" в смысле математическом) заполняются по простому закону.
// Я выделил эти два цикла из общего вложенного, чтобы не проверять i и j
// лишний раз и уменьшить диапазоны i и j во вложенном цикле.

for i in 1..(s1len+1)
{
    for j in 1..(s2len+1)
    {
        let m = match s1.chars().nth(i - 1) == s2.chars().nth(j - 1) {

```

```

        true => 0, // Совпадение
        false => 1, // Замена
    };

    d[i * (s2len + 1) + j] = min_of_n!(
        d[(i - 1) * (s2len + 1) + j] + 1, // Удаление
        d[i * (s2len + 1) + (j - 1)] + 1, // Вставка
        d[(i - 1) * (s2len + 1) + (j - 1)] + m) // Замена/Совпадение
    }
}

// print_matrix(&d, s2len + 1);

d[s1len * (s2len + 1) + s2len]
}

/// Функция находит расстояние Дамерау-Левенштейна упрощённым алгоритмом.

/// Это означает, что любая подстрока может быть отредактирована не более одного раза.

/// Это не всегда верно, если мы ищем названное расстояние в точности согласно его определению...

/// но достаточно хорошо во многих случаях :)
pub fn s_d_distance (s1: &str, s2: &str) -> usize
{

    let s1len = s1.chars().count();
    let s2len = s2.chars().count();

    if s1len == 0 && s2len == 0

```

```
{
    return 0;
}
else if s1len == 0 && s2len > 0
{
    return s2len;
}
else if s2len == 0
{
    return s1len;
}

let cell_count = (s1len + 1) * (s2len + 1); // Общее число ячеек матрицы
let mut d: Vec<usize> = vec![0; cell_count]; // Матрица D в виде одномерного
вектора

for i in 0..(s1len+1)
{
    d[i * (s2len + 1)] = i;
}

for j in 1..(s2len+1)
{
    d[j] = j;
}

// В отличие от расстояния Левенштейна мы также учитываем возможность
транспозиции двух соседних символов
```



```

for i in 1..(s1len+1)
{
  for j in 1..(s2len+1)
  {
    let m = match s1.chars().nth(i - 1) == s2.chars().nth(j - 1) {
      true => 0, // Совпадение
      false => 1, // Замена
    };

    d[i * (s2len + 1) + j] = min_of_n!(
      d[(i - 1) * (s2len + 1) + j] + 1, // Удаление
      d[i * (s2len + 1) + (j - 1)] + 1, // Вставка
      d[(i - 1) * (s2len + 1) + (j - 1)] + m); // Замена/Совпадение
    if i > 1 && j > 1 &&
      s1.chars().nth(i) == s2.chars().nth(j - 1) &&
      s1.chars().nth(i - 1) == s2.chars().nth(j)
    {
      d[i * (s2len + 1) + j] = min_of_n!(
        d[i * (s2len + 1) + j],
        d[(i - 2) * (s2len + 1) + (j - 2)] + 1 // Транспозиция
      )
    }
  }
}

d[s1len * (s2len + 1) + s2len]
}

```

/// Вывод матрицы, хранящейся как одномерный массив. Использовалось для тестирования.

```
#[allow(dead_code)]
```

```
fn print_matrix(m: &Vec<usize>, colcnt: usize)
```

```
{
```

```
    let mut testout = String::new();
```

```
    let mut pos = 0;
```

```
    for i in m
```

```
    {
```

```
        pos = pos + 1;
```

```
        testout.push_str(&(i.to_string()));
```

```
        testout.push(' ');
```

```
        if pos % (colcnt) == 0
```

```
        {
```

```
            println!("{}", testout);
```

```
            testout.clear();
```

```
        }
```

```
    }
```

```
}
```

```
#[cfg(test)]
```

```
mod tests {
```

```
    use super::*;
```

```
    #[test]
```

```
    fn levitest() {
```

```
        let result = l_distance("sitting", "kitten");
```

```

    assert_eq!(result, 3);

    let result = l_distance("polynomial", "exponential");

    assert_eq!(result, 6);
}

#[test]
fn sdametest() {
    let result = s_d_distance("ca", "abc");

    assert_eq!(result, 3);
}

#[test]
fn mintest() {
    let result = min_of_n!(545453,324356,2456);

    assert_eq!(result, 2456);

    let result = min_of_n!(2);

    assert_eq!(result, 2);
}
}

```

Экранные формы с примерами выполнения программы

Файл «main.py»:

```

Ковер
Диван для отдыха
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха', 'price': 5300}
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>

```

Файл «gen_random.py»:

```

[2, 1, 2, 2, 3]
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>

```

Файл «unique.py»:

```
a
A
b
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «sort.py»:

```
[123, 100, -100, -30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 4, -4, 1, -1, 0]
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «print_result.py»:

```
!!!!!!!
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «cm_timer.py»:

```
time: 5.500240087509155
time: 5.500479221343994
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «process_data.py»:

```
f1
<unique.Unique object at 0x000001BDA8F58D70>
f2
<filter object at 0x000001BDA6BF0AC0>
f3
<map object at 0x000001BDA6BF0AF0>
f4
программист 1С с опытом Python = 107901
программист с опытом Python = 151448
педагог программист с опытом Python = 111491
инженер-программист с опытом Python = 175170
инженер - программист с опытом Python = 136678
веб-программист с опытом Python = 189994
Старший программист с опытом Python = 158592
Системный программист (С, Linux) с опытом Python = 115564
Помощник веб-программиста с опытом Python = 143666
Инженер-электронщик (программист АСУ ТП) с опытом Python = 194821
Инженер-программист САПОУ (java) с опытом Python = 105093
Инженер-программист ПЛИС с опытом Python = 185316
Инженер-программист ККТ с опытом Python = 103439
Инженер-программист 1 категории с опытом Python = 113477
Инженер-программист (Орехово-Зуевский филиал) с опытом Python = 129087
Инженер-программист (Клинский филиал) с опытом Python = 135418
Инженер-программист с опытом Python = 134203
Инженер - программист АСУ ТП с опытом Python = 128266
Ведущий программист с опытом Python = 156572
Ведущий инженер-программист с опытом Python = 165404
Веб-программист с опытом Python = 124528
Веб - программист (PHP, JS) / Web разработчик с опытом Python = 196574
Web-программист с опытом Python = 166916
1С программист с опытом Python = 118834
time: 0.05558466911315918
PS C:\Users\User\Documents\BiquadraticEquations\PCPL> 
```